

FINAL PROJECT



Swan store

backend / data base / design

A MODERN BOUTIQUE MANAGEMENT SYSTEM: ELEVATING RETAIL OPERATIONS
THROUGH DATA EXCELLENCE



Swan store

Goal



The Problem

Boutiques often struggle with manual stock tracking, disconnected customer data, and inefficient loyalty management, leading to lost revenue and poor user experience.

The Solution

Swan Store provides a real-time management hub. From catalog updates to automated loyalty calculations, the system ensures data consistency and luxury service.

The Technology Stack 2

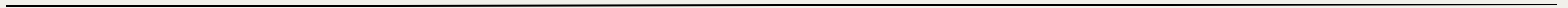
NODE.JS



PGADMIN 4



PostgreSQL

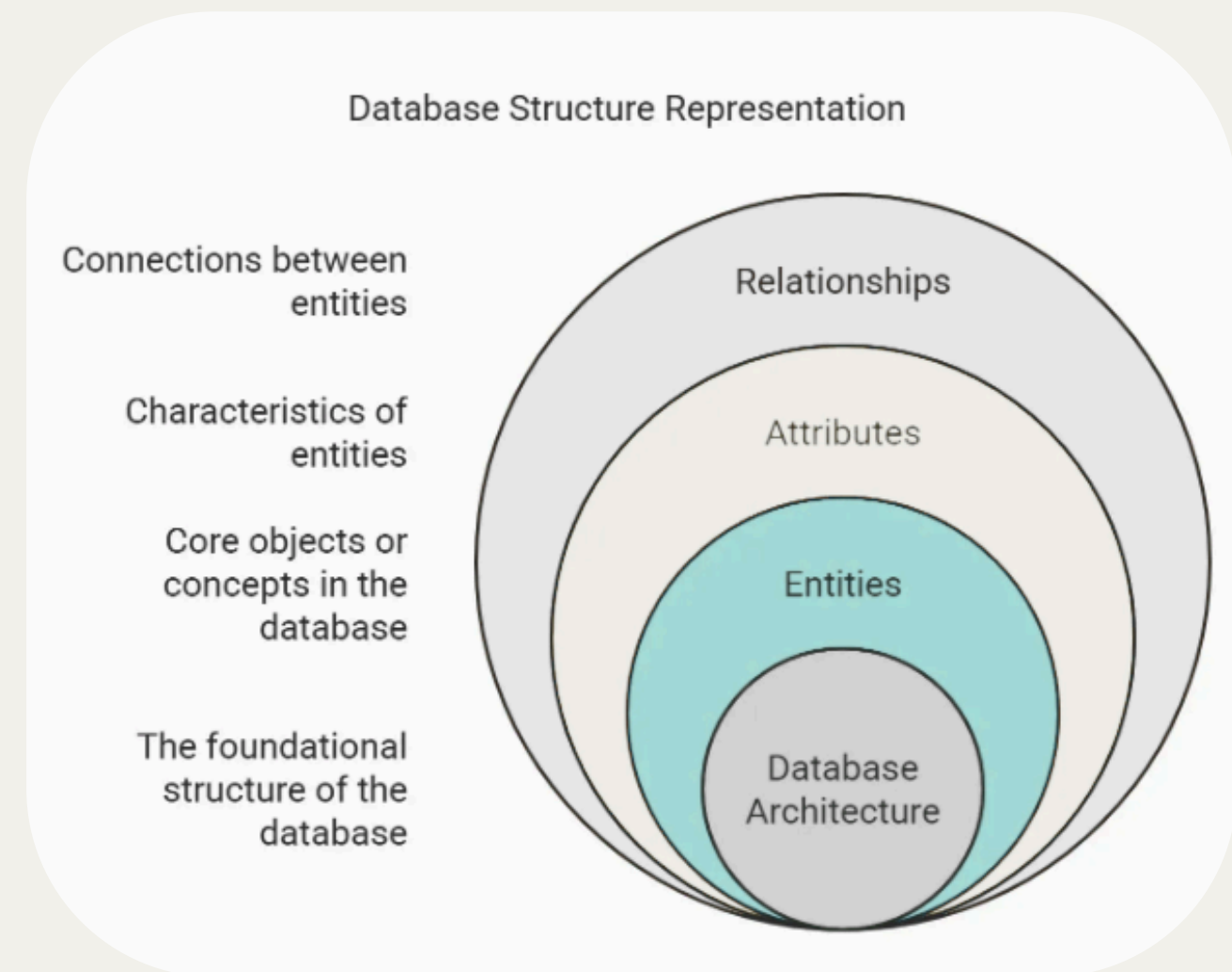


2 Entity Relationship Mapping

Relational Design

The schema centers around the Product-Order-Customer triad. We implemented a strictly normalized structure to eliminate redundancy.

- *Many-to-One: Products to Categories.*
- *One-to-Many: Customers to Orders.*
- *One-to-One: Loyalty Cards per Customer.*





System Schema Definition

Table Name	Primary Keys	Core Relationships	Key Constraint
Categories	id	None	NOT NULL Name
Products	id	category_id (FK)	ON DELETE SET NULL
Orders	id	customer_id (FK)	ON DELETE CASCADE
Loyalty Cards	id	customer_id (FK)	UNIQUE Customer

Project Assets & Links



Demo video



Certification



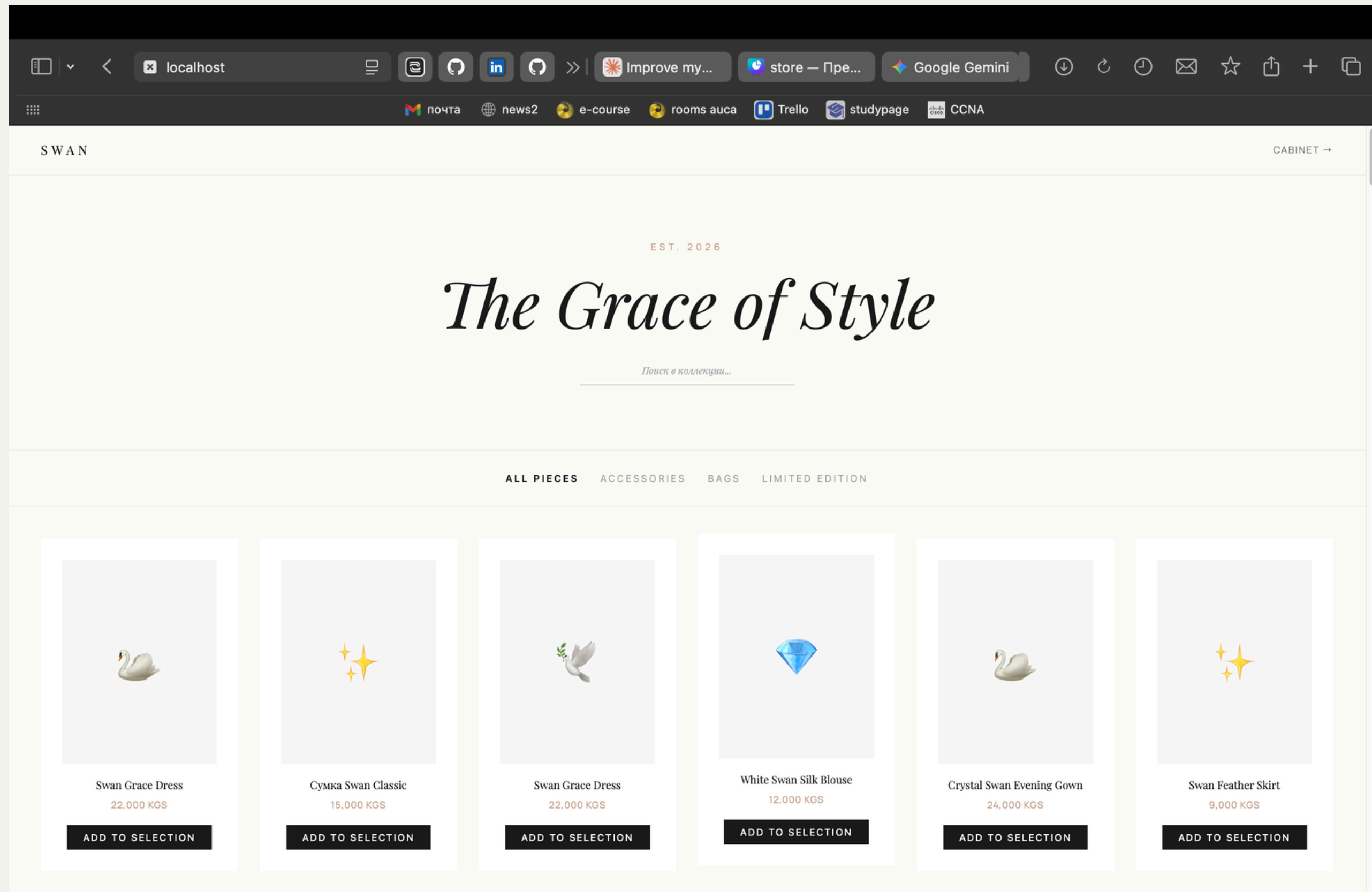
Repository



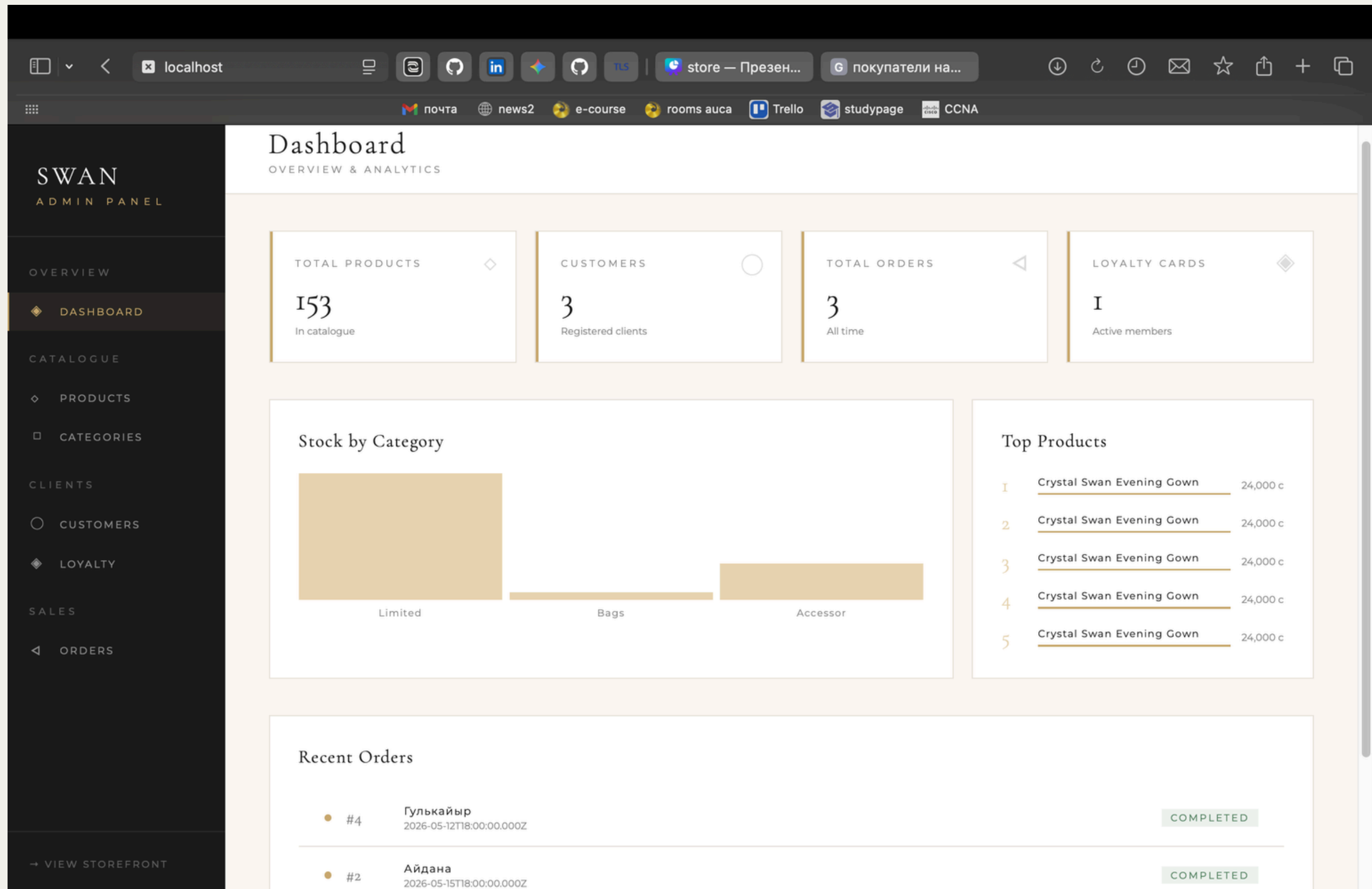
Screenshots



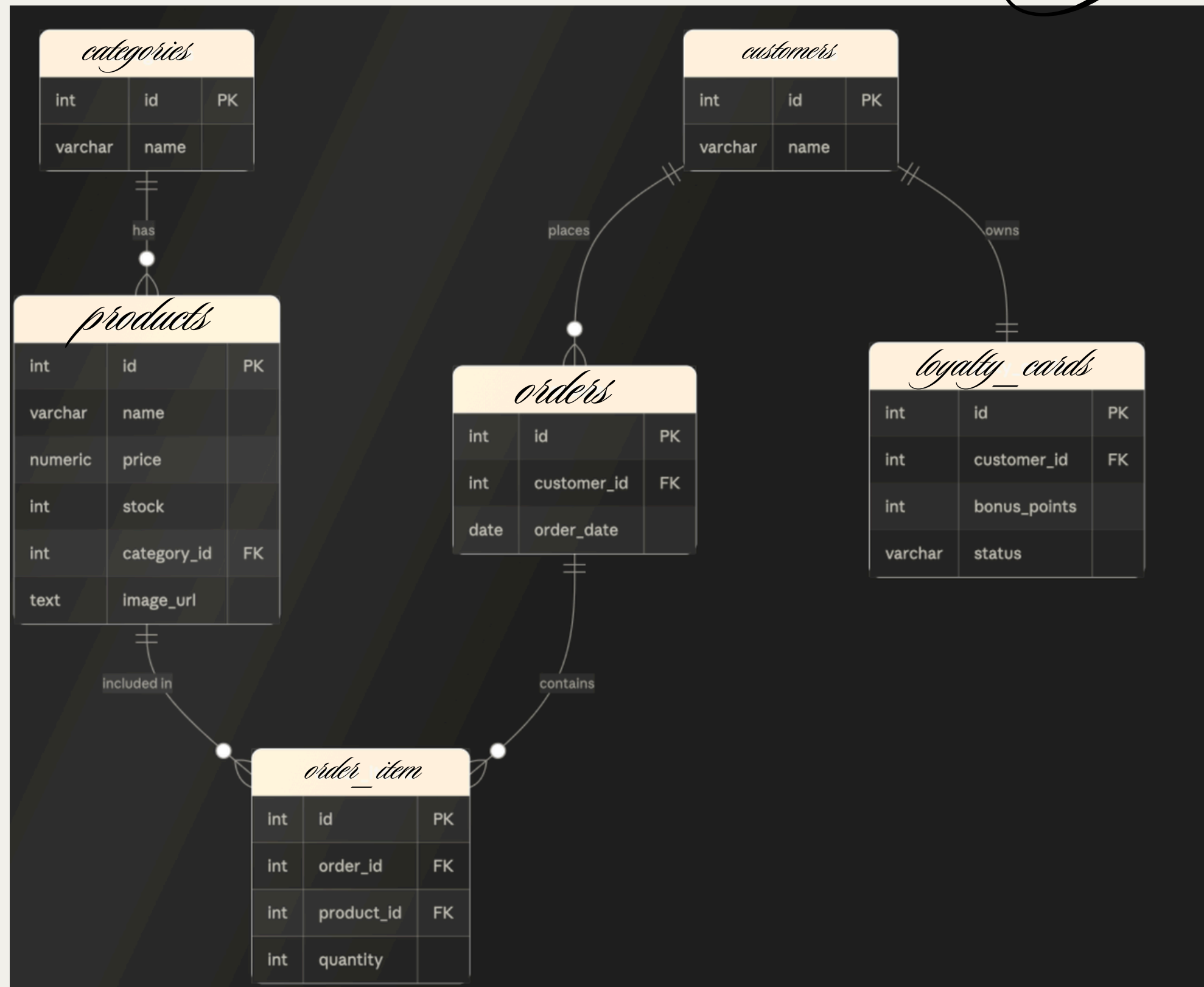
web site for customers



admin



ERD, schema design





pgAdmin Query Results



Query: Stock by Category

Categories

MANAGE COLLECTIONS

Add Category

CATEGORY NAME

e.g. Evening Wear

ADD CATEGORY

ALL CATEGORIES		
ID	NAME	ACTION
2	Accessories	<button>Remove</button>
1	Bags	<button>Remove</button>
3	Limited Edition	<button>Remove</button>

Aggregating inventory allows us to identify which product segments are overstocked or need replenishment.

```
SELECT c.name, SUM(p.stock)
FROM categories c
JOIN products p ON c.id = p.category_id
GROUP BY c.name;
```

Query: Top Premium Items

Top Products

1	<u>Crystal Swan Evening Gown</u>	24,000 c
2	<u>Crystal Swan Evening Gown</u>	24,000 c
3	<u>Crystal Swan Evening Gown</u>	24,000 c
4	<u>Crystal Swan Evening Gown</u>	24,000 c
5	<u>Crystal Swan Evening Gown</u>	

Identifying high-value products using ORDER BY and LIMIT clauses in pgAdmin.

```
SELECT name, price FROM products ORDER BY price DESC LIMIT 5;
```

Loyalty Program Engagement

Loyalty
MEMBERSHIP PROGRAM

Add Loyalty Card

CUSTOMER ID

1

BONUS POINTS

0

STATUS

Active

ADD CARD

LOYALTY MEMBERS

ID	CUSTOMER	POINTS	STATUS	ACTION
1	Customer #1	500 pts	SILVER	Remove
5	Customer #2	2340 pts	ACTIVE	Remove

LOYALTY CARDS

I

Active members

```
SELECT cu.name, lc.bonus_points
FROM loyalty_cards lc
JOIN customers cu ON lc.customer_id = cu.id
WHERE lc.status = 'active';
```

Query: Full Order Audit

Add New Product

PRODUCT NAME

e.g. Swan Grace Dress

PRICE (COM)

0

STOCK QUANTITY

0

CATEGORY ID

1

PRODUCT IMAGE

Choose File no file selected

ADD PRODUCT

A complex 4-table join executed in pgAdmin to produce a comprehensive sales report including customer names and itemized prices.

```
SELECT o.id, cu.name, p.name as product, oi.quantity,  
p.price  
FROM orders o  
JOIN customers cu ON o.customer_id = cu.id  
JOIN order_items oi ON o.id = oi.order_id  
JOIN products p ON oi.product_id = p.id;
```

Questions?

2 *Thank
You*